BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Lexical analysis.

Q2. In its left subtree (under the chosen no-duplicate convention).

Q3. Its element locations can be calculated from base address and index.

Q4. Sum 0 and carry-out 1.

You Do pages · Independent questions

Q1. Linker resolves external references/builds runnable image [3]; loader places executable/dependencies in memory and initiates process [3].

Q2. Static includes library code in binary [2], dynamic references shared library at runtime [2], size/updates/dependency trade-off [2].

Q3. Compile per module [2]; link/reference external dependency [2]; runtime loader requires compatible library [2]; mitigation packaging/version control [2].

Exit Ticket · Model answer

False. Dependencies may be missing/incompatible when loaded.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed:

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

LESSON 116 · Lesson 116 · Linkers, Loaders and Program Libraries

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. Which compilation stage converts source characters into tokens?

Q2. In a BST, where is a value less than the current node placed?

Q3. Why can an array provide direct indexed access?

Q4. A full adder receives A=1, B=1 and carry-in=0. What are sum and carry-out?

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

linker — Resolves symbols and combines object modules.

loader — Places an executable in memory ready to run.

library — Reusable compiled routines/functions.

dynamic linking — Resolution/use of shared code at load time or runtime.

New learning

Compile modules — Source files become object modules with symbols/references.

Link dependencies — Linker resolves symbols and combines modules/libraries; static/dynamic linkage differs.

Load and run — Loader maps executable and needed libraries into memory, then begins execution.

Worked example

main.c calls a function in maths.c. Explain the steps from separate source files to a running program.

1. Model answer — Compile source modules into object files; linker resolves the call and produces linked executable (or dynamic references); loader places executable and required libraries in memory and starts execution. — Check: link each claim to the scenario and use precise OCR terminology.

We Do · Practise with guidance

A program compiles but the linker reports 'undefined reference: calculateTax'. Diagnose the fault.

What must we decide?

Identify the inputs, relevant facts and exact output or judgement required.

Work through the method

Use the middle stage of the visual to follow the process and state each intermediate result.

Shared worked answer

Lexical and syntax stages succeeded. The linker cannot find the referenced symbol's definition: implementation may be missing, not linked, misspelled or incompatible. Add the correct object/library or fix the symbol.

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

Explain the roles of linker and loader with examples. [6]

Question 2

Distinguish static from dynamic linking, giving one trade-off of each. [6]

Question 3

A deployed app fails because a required shared library is unavailable. Explain why compile success did not prevent this. [8]

Exit Ticket · Answer before checking

A successful compilation guarantees that every dynamically linked library will be available at runtime.